hackathon
GENERATIVE AI

Challenge proposé par :

IBM

Équipe :

Hagop Hannachian
Yanis Hammaci
Yacine Tamdrari
Lyes Chalal

Lien Git : [Git]

Du 18 au 27 mars 2026

UNIVERSITÉ PARIS 8 DES CRÉATIONS

université PARIS-SACLAY

UQAC Université du Québec à Chicoutimi

Table des matières

1 Pertinence et innovation **3**

2 Qualité technique et faisabilité **3**

3 Impact, risques et responsabilité **5**

4 Conclusion et recommandations **5**

1 Pertinence et innovation

Identification des besoins

Le développement logiciel moderne est confronté à une complexité croissante, liée à la multiplicité des outils, à l'accélération des cycles de livraison et à l'exigence constante de qualité. Les chiffres parlent d'eux-mêmes : les développeurs perdent jusqu'à 8 heures par semaine à cause d'inefficacités liées à la dette technique et aux mauvais processus [1]. Bien que de nombreuses solutions existent aujourd'hui générateurs de code [2], assistants IA, outils DevOps elles restent largement insuffisantes : 66% des suggestions de code produites par les outils IA actuels s'avèrent inexactes [3], 45% des utilisateurs rapportent un temps de débogage allongé[3], particulièrement sur de grandes codebases, et ces outils demeurent inadaptés aux tâches critiques comme le déploiement ou la planification de projet. Fragmentées et spécialisées sur une fraction du cycle de développement, elles nécessitent une forte intervention humaine pour assurer leur cohérence globale.

Dans ce contexte, l'automatisation des tâches répétitives et la structuration intelligente des processus de développement deviennent des enjeux majeurs. Le cas d'usage proposé dans ce hackathon s'inscrit pleinement dans cette problématique en visant à dépasser les limites des solutions existantes, en proposant une approche unifiée capable d'automatiser la génération d'un pipeline complet de livraison logicielle à partir d'une simple expression de besoin.

Cette couverture bout-en-bout répond à deux lacunes majeures des solutions existantes : d'une part, les assistants IA actuels manquent de spécialisation, traitant l'ensemble des tâches avec un agent généraliste incapable d'atteindre la profondeur requise sur chaque domaine ; d'autre part, les systèmes multi-agents existants peinent à maintenir une cohérence globale, souffrant de fragmentation du contexte et de perte de fidélité lors des échanges inter-agents. Notre architecture répond à ces défis en combinant des agents spécialisés (Architecte, Développeur, DevOps, QA, etc.) avec des canaux de communication structurés, garantissant la continuité du contexte et la cohérence entre les artefacts produits, tout en s'adaptant à différents types de projets et de contextes métiers, en offrant une solution autonome et fiable.

Face aux réalités actuelles et aux évolutions futures du développement logiciel, notre système s'intègre naturellement aux environnements opérationnels des équipes en combinant connaissances internes et sources d'information externes actualisées via un mécanisme de recherche guidée, garantissant des décisions alignées avec l'état de l'art. Il permet également l'intégration native de bibliothèques internes propres par l'utilisateur, produisant un code directement conforme aux standards existants et facilitant son adoption en contexte industriel. Enfin, son architecture se distingue par sa capacité d'adaptation : en fonction du domaine du projet, des agents spécialisés peuvent être mobilisés dynamiquement en complément des agents socles (Architecte, Développeur, DevOps, QA) par exemple, un projet dans le domaine de la cybersécurité verra automatiquement émerger des agents spécialisés sur ces enjeux, venant compléter les agents socles (Architecte, Développeur, DevOps, QA) toujours présents positionnant ainsi la solution comme un système flexible et évolutif, capable de s'aligner efficacement sur les contraintes métier.

2 Qualité technique et faisabilité

Une architecture dynamique

Actuellement, les architectures les plus utilisées pour les systèmes multi-agents [4] reposent sur un modèle séquentiel (workflow) où les agents : si le Développeur termine le code et que le QA détecte un problème, le système doit repartir du début du cycle et retraverser toutes les étapes. Dans notre cas, nous avons adopté une architecture de type swarm, où tous les agents opèrent en parallèle et sont interconnectés en permanence. Dans ce même scénario, le QA notifie directement le Développeur qui corrige immédiatement, sans interrompre le travail des autres agents.

Cependant, un swarm purement statique manque de flexibilité face à des problèmes imprévus ou des domaines d'expertise non anticipés. Notre architecture est donc dynamique : elle combine un socle d'agents permanents spécialisés, garants de la continuité du pipeline, avec une partie dynamique assurée par le Spawner [5] un agent dédié qui analyse en permanence le contexte partagé et instancie à la volée des agents spécialistes adaptés aux besoins détectés en cours d'exécution.

Des agents spécialisés

Pour qu'un pipeline de livraison logicielle soit fiable, chaque domaine doit être adressé par une expertise dédiée. Notre système repose sur un socle d'agents persistants couvrant l'intégralité du cycle : de l'analyse des spécifications jusqu'à la validation finale, en passant par la conception de l'architecture, la génération du code, la configuration de

1. https://www.baytechconsulting.com/blog/unlocking-developer-happiness-productivity-2025
2. https://survey.stackoverflow.co/2025/ai/
3. https://www.index.dev/blog/developer-productivity-statistics-with-ai-tools
4. https://code.claude.com/docs/en/agent-teams
5. https://www.anthropic.com/engineering/multi-agent-research-system

l'environnement et l'assurance qualité. Chaque agent est responsable de son périmètre, produit des livrables structurés.

La coordination repose sur trois canaux complémentaires. Le premier est la messagerie directe inter-agents : chaque agent peut envoyer un message ciblé à un autre pour le notifier, lui soumettre une question ou signaler un problème. Par exemple, l'Architecte peut notifier le Développeur qu'il n'a pas respecté un contrat d'interface, le QA peut demander au Développeur si le code est prêt avant de lancer les tests, ou encore remonter précisément les tests en échec pour qu'il les corrige. Le second est le blackboard partagé [6] : un espace commun où chaque agent publie en temps réel le contexte de ses actions. Lorsque le Développeur crée un fichier, il inscrit cette action sur le blackboard, permettant à tous les autres agents de suivre l'avancement du projet et d'adapter leur comportement en conséquence. Le troisième est l'espace de fichiers partagé : le lieu où sont déposés les livrables concrets produits par chaque agent comme le code source, tests, ...etc accessibles et consultables par l'ensemble de l'équipe à tout moment.

Une mémoire long terme

Pour que cette collaboration reste cohérente sur la durée, les agents ne peuvent pas reposer uniquement sur leur contexte immédiat. Notre système intègre une mémoire long terme hybride combinant une recherche sémantique par embeddings et une recherche lexicale (BM25), fusionnées par Reciprocal Rank Fusion. Chaque action significative livrable produit, décision prise, erreur rencontrée est automatiquement indexée et consultable par n'importe quel agent à tout moment. Par exemple, si le Développeur rencontre une erreur, il interroge la mémoire et retrouve qu'un bug similaire a déjà été rencontré et résolu lors d'une itération précédente, lui permettant d'appliquer directement la solution sans repartir de zéro.

Des tools pour interagir avec l'environnement

Les agents doivent pouvoir agir sur leur environnement. Chaque agent dispose d'un ensemble de tools couvrant les opérations essentielles : lire et écrire des fichiers, exécuter des commandes shell, naviguer dans la structure du projet. Au-delà de ces capacités de base, les agents disposent d'un outil de navigation web leur permettant d'interagir directement avec le monde extérieur rechercher de la documentation en ligne, consulter des références techniques à jour, ou explorer des ressources web en temps réel. Par exemple, le déployeur peut utiliser ce tool pour déployer directement l'application sur un environnement cible, tandis que le Développeur peut naviguer sur le web pour résoudre une erreur liée à une dépendance et appliquer immédiatement la solution trouvée.

Stach technique

Strands Agents [7] a été retenu comme framework agentique pour sa simplicité, sa modularité et son alignement direct avec notre architecture. Contrairement aux frameworks imposant des abstractions rigides, Strands tire parti des capacités de raisonnement des LLMs pour orchestrer de manière autonome la planification, l'enchaînement des tâches et l'appel aux tools. Il intègre nativement les patterns multi-agents Graph, Swarm, Workflow, etc. [8], ainsi qu'un support pour le Model Context Protocol (MCP) [9], permettant d'étendre les capacités de nos agents à des outils externes sans modifier le cœur du système. Strands propose également plus des tools pré-construits [10] couvrant la manipulation de fichiers, les requêtes API et l'interaction avec des services externes. La mémoire long terme est assurée par ChromaDB [11] comme base vectorielle, combinée à des modèles d'embeddings sentence-transformers [12] pour la recherche sémantique et à un index BM25 pour la recherche lexicale le tout fusionné par Reciprocal Rank Fusion pour un rappel hybride

Flexibilité des modèles et choix du fournisseur LLM

Notre architecture est conçue pour être agnostique au fournisseur de modèles : elle peut se connecter nativement à n'importe quel provider LLM Anthropic, OpenAI, Ollama, Mistral, Bedrock, Groq, et bien d'autres sans modifier le cœur du système. Dans le cadre de ce hackathon, nous avons opté pour Snowflake Cortex comme fournisseur LLM.

Supervision et interaction humaine Un tableau de bord en temps réel permet à l'utilisateur de superviser l'ensemble du pipeline (messages échangés entre agents, artefacts générés, état d'avancement de chaque agent, et traces de raisonnement, offrant une visibilité totale sur le comportement du système à chaque instant. L'utilisateur peut injecter des instructions directement en cours d'exécution sans interrompre ni redémarrer le processus. Cette capacité de steering permet de réorienter un agent sur une contrainte oubliée, de corriger une décision architecturale en cours de route, ou d'affiner le comportement du système face à un résultat inattendu.

Personnalisation et adaptation au contexte client

6. https://arxiv.org/html/2507.01701v1
7. https://strandsagents.com/0.1.x/
8. https://strandsagents.com/docs/user-guide/concepts/multi-agent/swarm/
9. https://strandsagents.com/docs/user-guide/concepts/tools/mcp-tools/
10. https://strandsagents.com/docs/user-guide/concepts/tools/community-tools-package/
11. https://www.trychroma.com/
12. https://huggingface.co/sentence-transformers

Notre solution est conçue pour s'adapter à l'environnement technologique de chaque organisation. Les bibliothèques de composants internes peuvent être intégrées sous forme de skills [13] en fournissant simplement leur documentation sous forme d'un document. Pour la charte graphique, notre système exploite des modèles de vision (VLM) capables d'analyser les images.

Intégration aux systèmes existants

Les agents peuvent consommer nativement tout outil ou service exposé via MCP qu'il s'agisse d'un service métier, d'un gestionnaire de dépendances ou d'un registre interne sans aucune modification du cœur du système. D'autre part, notre solution peut elle-même être exposée comme un service MCP.

3 Impact, risques et responsabilité

Identification des principaux risques

Le premier est le risque de génération de code incorrect ou incomplet : malgré les boucles de validation entre agents, un LLM peut produire du code fonctionnellement erroné sans que les tests ne le détectent. Une dégradation des performances du LLM sous-jacent impacte directement la qualité de l'ensemble des livrables produits.

Analyse des impacts

En contexte entreprise, un code généré sans validation humaine suffisante peut introduire des failles de sécurité [14] non détectées, des régressions fonctionnelles ou des incompatibilités avec les systèmes existants. Par ailleurs, l'automatisation poussée du pipeline peut créer une fausse impression de complétude : les artefacts générés sont structurellement cohérents mais peuvent ne pas refléter fidèlement l'intention métier réelle du client, notamment sur des spécifications ambiguës ou incomplètes.

Mesures concrètes pour limiter les risques

Pour contenir ces risques, notre système intègre plusieurs mécanismes concrets. L'agent reviewer vérifie la cohérence entre l'ensemble des livrables produits spécifications, architecture, code et va jusqu'à exécuter l'application pour valider son bon fonctionnement avant d'émettre un verdict, garantissant ainsi qu'aucun PASS ne soit délivré sur un livrable non opérationnel. L'agent QA impose un seuil minimal de couverture de tests et remonte les échecs directement au Développeur pour correction immédiate. Enfin, en cas de blocage répété, le Spawner instancie dynamiquement un agent spécialiste adapté pour résoudre le problème.

Prise en compte du cadre réglementaire

L'utilisation exclusive de modèles et frameworks open source garantit qu'aucune donnée sensible ne transite par des services tiers non maîtrisés un prérequis dans des secteurs réglementés comme la banque ou la santé. L'exposition des traces de raisonnement répond par ailleurs aux exigences d'explicabilité et d'auditabilité de l'AI Act européen [15].

4 Conclusion et recommandations

Synthèse des apports

Notre solution démontre qu'il est possible de transformer une simple expression de besoin en un pipeline de livraison logicielle complet et opérationnel, grâce à une architecture multi-agents alliant spécialisation et adaptabilité dynamique. Elle s'intègre naturellement dans des environnements entreprise hétérogènes via la compatibilité MCP, le support des bibliothèques internes et l'adaptation aux chartes graphiques clients. Au-delà de la simple génération de code, le système livre une architecture documentée, un environnement de développement configuré, des tests validés et une configuration de déploiement prête à l'emploi constituant un socle immédiatement exploitable par n'importe quelle équipe de développement.

Limites identifiées

La qualité des livrables reste dépendante de la précision des spécifications fournies en entrée : une expression de besoin ambiguë ou incomplète se répercute directement sur l'ensemble des artefacts générés. Par ailleurs, le choix des LLM influence la qualité des artifacts génerés. Enfin, une relecture humaine reste nécessaire sur des projets à fort enjeu métier ou dans des domaines nécessitant une expertise métier pointue que les agents ne peuvent pas inférer des seules spécifications.

déploiement et perspectives d'évolution

13. https://strandsagents.com/docs/user-guide/concepts/plugins/skills/
14. https://arxiv.org/html/2506.04133v3
15. https://artificialintelligenceact.eu/

Le passage à des modèles de raisonnement plus avancés pour les agents à forte charge cognitive comme l'Architecte et le Reviewer améliorerait la qualité des décisions techniques prises de manière autonome. Enfin, l'enrichissement continu de la bibliothèque de skills sectoriels cybersécurité, finance, santé préconfigurés et validés par des experts métier permettrait d'adresser des cas d'usage à plus forte valeur ajoutée.

À moyen terme, le déploiement en production à grande échelle soulève des défis spécifiques qui doivent être anticipés. La conteneurisation du système permettrait un déploiement distribué sur des infrastructures cloud avec une montée en charge élastique selon la complexité des projets traités. La construction d'un système de monitoring dédié suivi des coûts de tokens, des temps de réponse par agent et des taux d'erreurs (par agent, tool, etc.), permettrait aux équipes de piloter la solution en production et d'identifier les goulets d'étranglement avant qu'ils n'impactent les livraisons.